

79. Given a collection of numbers, nums, that might contain duplicates, return all possible unique permutations in any order. Example 1: Input: nums = [1,1,2] Output: [[1,1,2], [1,2,1], [2,1,1]] Example 2: Input: nums = [1,2,3] Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

```
def permuteUnique(nums):
    nums.sort()
    res=[]
    used=[False]*len(nums)

    def dfs(path):
        if len(path)==len(nums):
            res.append(path[:])
            return

        for i in range(len(nums)):
            if used[i]:
                continue

            if i>0 and nums[i]==nums[i-1] and not used[i-1]:
                continue

            used[i]=True
            dfs(path+[nums[i]])
            used[i]=False

    dfs([])
    return res

print(permuteUnique([1,1,2]))
```

```
*** [[1, 1, 2], [1, 2, 1], [2, 1, 1]]
```

78. Given an array `nums` of distinct integers, return all the possible permutations. You can return the answer in any order. Example 1: Input: `nums = [1,2,3]` Output: `[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]` Example 2: Input: `nums = [0,1]` Output: `[[0,1],[1,0]]` Example 3: Input: `nums = [1]` Output: `[[1]]`

```
def permute(nums):
    res=[]

    def dfs(path,remain):
        if not remain:
            res.append(path)
            return

        for i in range(len(remain)):
            dfs(path+[remain[i]],
                remain[:i]+remain[i+1:])

    dfs([],nums)
    return res

print(permute([1,2,3]))
```

```
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
```

77. Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target. Each number in candidates may only be used once in the combination. The solution set must not contain duplicate combinations. Example 1: Input: candidates = [10,1,2,7,6,1,5], target = 8 Output: [[1,1,6], [1,2,5], [1,7], [2,6]] Example 2: Input: candidates = [2,5,2,1,2], target = 5 Output: [[1,2,2], [5]]

```
def combinationSum2(candidates, target):
    candidates.sort()
    res=[]

    def dfs(start, target, path):
        if target==0:
            res.append(path[:])
            return

        for i in range(start, len(candidates)):
            if i>start and candidates[i]==candidates[i-1]:
                continue

            if candidates[i]>target:
                break

            dfs(i+1,
                target-candidates[i],
                path+[candidates[i]])

    dfs(0, target, [])
    return res

print(combinationSum2(
    [10,1,2,7,6,1,5],8))
```

```
[[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]
```

76. Given an array of distinct integers `candidates` and a target integer `target`, return a list of all unique combinations of `candidates` where the chosen numbers sum to `target`. You may return the combinations in any order. The same number may be chosen from `candidates` an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different. The test cases are generated such that the number of unique combinations that sum up to `target` is less than 150 combinations for the given input. Example 1: Input: `candidates = [2,3,6,7]`, `target = 7` Output: `[[2,2,3],[7]]` Explanation: 2 and 3 are candidates, and $2 + 2 + 3 = 7$. Note that 2 can be used multiple times. 7 is a candidate, and $7 = 7$. These are the only two combinations. Example 2: Input: `candidates = [2,3,5]`, `target = 8` Output: `[[2,2,2,2],[2,3,3],[3,5]]`

COMBINATION SUM 2:

```
def combinationSum(candidates, target):
    res = []

    def dfs(start, target, path):
        if target == 0:
            res.append(path[:])
            return

        for i in range(start, len(candidates)):
            if candidates[i] <= target:
                dfs(i, target - candidates[i],
                    path + [candidates[i]])

    dfs(0, target, [])
    return res

print(combinationSum([2,3,6,7], 7))
```

```
[[2, 2, 3], [7]]
```

75. Given an array of integers `arr`, find the sum of $\min(b)$, where b ranges over every (contiguous) subarray of `arr`. Since the answer may be large, return the answer modulo 109 +

Example 1: Input: `arr = [3,1,2,4]` Output: 17 Explanation: Subarrays are [3], [1], [2], [4], [3,1], [1,2], [2,4], [3,1,2], [1,2,4], [3,1,2,4]. Minimums are 3, 1, 2, 4, 1, 1, 2, 1, 1, 1. Sum is 17. Example 2: Input: `arr = [11,81,94,43,3]` Output: 444

```
def sumSubarrayMins(arr):
    MOD=10**9+7
    stack=[]
    ans=0

    for i in range(len(arr)+1):
        cur=arr[i] if i<len(arr) else 0

        while stack and arr[stack[-1]]>cur:
            mid=stack.pop()

            left=stack[-1] if stack else -1
            right=i

            ans+=arr[mid]*(mid-left)*(right-mid)

        stack.append(i)

    return ans%MOD

print(sumSubarrayMins([3,1,2,4]))
```

74. You are given an integer array `nums` and an integer `target`. You want to build an expression out of `nums` by adding one of the symbols '+' and '-' before each integer in `nums` and then

concatenate all the integers. For example, if `nums = [2, 1]`, you can add a '+' before 2 and a '-' before 1 and concatenate them to build the expression "+2-1". Return the number of different expressions that you can build, which evaluates to `target`. Example 1: Input: `nums = [1,1,1,1,1]`, `target = 3` Output: 5 Explanation: There are 5 ways to assign symbols to make the sum of `nums` be `target` 3. -1 + 1 + 1 + 1 + 1 = 3 +1 - 1 + 1 + 1 + 1 = 3 +1 + 1 - 1 + 1 + 1 = 3 +1 + 1 + 1 - 1 + 1 = 3 +1 + 1 + 1 + 1 - 1 = 3 Example 2: Input: `nums = [1]`, `target = 1` Output: 1

```
def findTargetSumWays(nums, target):
    dp={0:1}

    for num in nums:
        nxt={}
        for s,cnt in dp.items():
            nxt[s+num]=nxt.get(s+num,0)+cnt
            nxt[s-num]=nxt.get(s-num,0)+cnt
        dp=nxt

    return dp.get(target,0)

print(findTargetSumWays([1,1,1,1,1],3))
```


73. Write a program to solve a Sudoku puzzle by filling the empty cells. A sudoku solution must satisfy all of the following rules: Each of the digits 1-9 must occur exactly once in each row. Each of the digits 1-9 must occur exactly once in each column. Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid. The '.' character indicates empty cells. Example 1: Input: board = [["5","3",".",,"7",".",,".",,"."], ["6",".",,".",,"1","9","5",".",,"."], [".","9","8",".",,".",,".",,"6","."], ["8",".",,".",,".",,"6",".",,".",,"3"], ["4",".",,".",,"8",".",,"3",".",,".",,"1"], ["7",".",,".",,".",,".",,".",,"2",".",,"6"], [".","6",".",,".",,".",,".",,"2","8","."], [".",".",,".",,"4","1","9",".",,".",,"5"], [".",".",,".",,"8",".",,".",,"7","9","."]] Output: [["5","3","4","6","7","8","9","1","2"], ["6","7","2","1","9","5","3","4","8"], ["1","9","8","3","4","2","5","6","7"], ["8","5","9","7","6","1","4","2","3"], ["4","2","6","8","5","3","7","9","1"], ["7","1","3","9","2","4","8","5","6"], ["9","6","1","5","3","7","2","8","4"], ["2","8","7","4","1","9","6","3","5"], ["3","4","5","2","8","6","1","7","9"]]

```
def solveSudoku(board):

    def valid(r,c,num):
        for i in range(9):
            if board[r][i]==num:
                return False
            if board[i][c]==num:
                return False

        sr=(r//3)*3
        sc=(c//3)*3

        for i in range(sr,sr+3):
            for j in range(sc,sc+3):
                if board[i][j]==num:
                    return False
        return True

    def solve():
        for r in range(9):
            for c in range(9):
                if board[r][c]=='.':
                    for num in '123456789':
                        if valid(r,c,num):
                            board[r][c]=num
                            if solve():
                                return True
                            board[r][c]='.'
                    return False
        return True

    solve()
    return board
```

72. Write a program to solve a Sudoku puzzle by filling the empty cells. A sudoku solution must satisfy all of the following rules: Each of the digits 1-9 must occur exactly once in each row. Each of the digits 1-9 must occur exactly once in each column. Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid. The '.' character indicates empty cells. Example 1: Input:

board = [["5","3",".",,".",,"7",".",,".",,"."], ["6",".",,".",,"1","9","5",".",,"."],

[["9","8",".",,".",,"6",".",,"."], ["8",".",,".",,"6",".",,".",,"3"], ["4",".",,".",,"8",".",,"3",".",,"1"], ["7",".",,".",,"2",".",,".",,".",,"6"], [".","6",".",,".",,".",,".",,"2","8","."],
[[".",,".",,"4","1","9",".",,".",,"5"], [".",".",,".",,"8",".",,".",,"7","9"], ["5",".",,".",,".",,".",,".",,".",,"."]] Output: [["5","3","4","6","7","8","9","1","2"], ["6","7","2","1","9","5","3","4","8"],
["1","9","8","3","4","2","5","6","7"], ["8","5","9","7","6","1","4","2","3"], ["4","2","6","8","5","3","7","9","1"], ["7","1","3","9","2","4","8","5","6"],
["9","6","1","5","3","7","2","8","4"], ["2","8","7","4","1","9","6","3","5"], ["3","4","5","2","8","6","1","7","9"]]

```
def solveSudoku(board):

    def valid(r,c,num):
        for i in range(9):
            if board[r][i]==num:
                return False
            if board[i][c]==num:
                return False

        sr=(r//3)*3
        sc=(c//3)*3

        for i in range(sr,sr+3):
            for j in range(sc,sc+3):
                if board[i][j]==num:
                    return False
        return True

    def solve():
        for r in range(9):
            for c in range(9):
                if board[r][c]=='.':
                    for num in '123456789':
                        if valid(r,c,num):
                            board[r][c]=num
                            if solve():
                                return True
                            board[r][c]='.'
                    return False
        return True

    solve()
    return board
```


71. Discuss the generalization of the N-Queens Problem to other board sizes and shapes, such as rectangular boards or boards with obstacles. Explain how the algorithm can be adapted to handle these variations and the additional constraints they introduce. Provide examples of solving generalized N-Queens Problems for different board configurations, such as an 8×10 board, a 5×5 board with obstacles, and a 6×6 board with restricted positions. a. 8×10 Board: 8 rows and 10 columns Output: Possible solution [1, 3, 5, 7, 9, 2, 4, 6] Explanation: Adapt the algorithm to place 8 queens on an 8×10 board, ensuring no two queens threaten each other. b. 5×5 Board with Obstacles: Input: N = 5, Obstacles at positions [(2, 2), (4, 4)] Output: Possible solution [1, 3, 5, 2, 4] Explanation: Modify the algorithm to avoid placing queens on obstacle positions, ensuring a valid solution that respects the constraints. c. 6×6 Board with Restricted Positions: Input: N = 6, Restricted positions at columns 2 and 4 for the first queen Output: Possible solution [1, 3, 5, 2, 4, 6] Explanation: Adjust the algorithm to handle restricted positions, ensuring the queens are placed without conflicts and within allowed columns.

```
def solve(n):
    res=[]
    col=set()
    d1=set()
    d2=set()

    def bt(r,path):
        if r==n:
            res.append(path[:])
            return

        for c in range(n):
            if c in col or r-c in d1 or r+c in d2:
                continue

            col.add(c)
            d1.add(r-c)
            d2.add(r+c)

            bt(r+1,path+[c+1])

            col.remove(c)
            d1.remove(r-c)
            d2.remove(r+c)

    bt(0,[])
    print(res[0])

solve(5)
```

[1, 3, 5, 2, 4]

CHAPTER 6

70. Discuss the importance of visualizing the solutions of the N-Queens Problem to understand the placement of queens better. Use a graphical representation to show how queens are placed on the board for different values of N. Explain how visual tools can help in debugging the algorithm and gaining insights into the problem's complexity. Provide examples of visual representations for N = 4, N = 5, and N = 8, showing different valid solutions. a. Visualization for 4-Queens: Input: N = 4 Output: Explanation: Each 'Q' represents a queen, and '.' represents an empty space. b. Visualization for 5-Queens: Input: N = 5 Output: c. Visualization for 8-Queens: Input: N = 8 Output:

```
def solveNQueens(n):
    col=set()
    diag1=set()
    diag2=set()
    board=[['.']*n for _ in range(n)]

    def backtrack(r):
        if r==n:
            for row in board:
                print(' '.join(row))
            return True

        for c in range(n):
            if c in col or r-c in diag1 or r+c in diag2:
                continue

            col.add(c)
            diag1.add(r-c)
            diag2.add(r+c)
            board[r][c]='Q'

            if backtrack(r+1):
                return True

            col.remove(c)
            diag1.remove(r-c)
            diag2.remove(r+c)
            board[r][c]='.'

    backtrack(0)

solveNQueens(4)
```

```
...
. Q . .
. . . Q
Q . . .
. . Q .
```